

Flutter Core Project – Technical Interview Review (Chi Tiet)

Tai lieu tong hop ky thuat CHI TIET cho project flutter_core_project (MyTHP Internal App).

Bao gom code snippets thuc te, API schema, data flow chi tiet, phan tich tung pattern de chuan bi phong van technical.

MUC LUC

Elevator Pitch
Kien truc tong quan & Folder Structure
Design Patterns chi tiet
Startup Flow - Phan tich tung dong
Dependency Injection (GetIt)
Networking - Dio + Interceptors + Error Handling
State Management - BLoC & HydratedCubit
Data Layer - Models, Repositories, API Services
Firebase - FCM, Analytics, Crashlytics
Work Schedule & Local Notifications
Authentication & Session
Localization & Theme Persistence
Feature Inventory day du
Third-party Libraries Matrix
Platform Config - Android & iOS
Code Quality & Technical Debt
Interview Q&A (30+ cau hoi)
File Map quan trong
Checklist on tap
Build & Run Scripts

1. ELEVATOR PITCH

MyTHP la app Flutter noi bo doanh nghiep (HR/Attendance), target nhan vien cong ty.

Tinh nang chinh:

- Auth: Dang nhap noi bo (username/password) + JWT token session
- Timesheet (Bang cong): Xem cong theo thang, navigation thang truoc/sau, cache thong minh, xem chi tiet tung ngay voi diem cham cong
- Adjustment Report: Gui yeu cau dieu chinh cong qua API
- Request History: Lich su cac yeu cau da gui (phan trang)
- Notification Center: Danh sach thong bao, danh dau da doc, unread badge
- FCM: Nhan push notification tu backend, register token
- Work Schedule: Cai dat ca lam, nhac check-in/check-out/canh bao tre/overtime qua local notification
- Da ngon ngu: Tieng Viet / Tieng Anh (JSON-based localization)
- Dark Mode: Persist qua HydratedCubit
- Analytics + Crashlytics: Firebase screen tracking + error monitoring

Stack chinh: Flutter + Clean Architecture + BLoC + GetIt DI + Dio + Firebase

2. KIEN TRUC TONG QUAN & FOLDER STRUCTURE

2.1 Folder Structure day du

```
lib/
|-- main.dart          # Entrypoint production (default)
|-- main_dev.dart      # Entrypoint dev flavor (.env.dev)
|-- main_prod.dart     # Entrypoint prod flavor (.env.prod)
|-- firebase_options.dart # Auto-generated Firebase config
|-- injection_container.dart # GetIt DI wiring toan app
|
|-- core/
| |-- configs/
| | |-- app_config.dart # Doc .env: baseUrl, timeout, title, env
| | |-- api_error_config.dart # Model: ApiErrorDialogConfig
| | +--- theme/
| | | |-- app_theme.dart # ThemeData light + dark
| | | |-- app_colors.dart # Color palette constants
| | +--- app_text_styles.dart
|
|-- constants/
| +--- constants.dart # newsAPIBaseUrl, newsAPIKey (hardcoded - TECHNICAL DEBT)
|
|-- data/
| |-- data_sources/remote/
| | |-- login_api_service.dart
| | |-- timesheet_api_service.dart
| | |-- notification_api_service.dart
| | |-- adjustment_report_api_service.dart
| | |-- request_history_api_service.dart
| | +--- news_api_service.dart # Retrofit-generated
| |-- models/
| | |-- auth/login_model.dart
| | |-- timesheet/timesheet_model.dart
| | |-- notification/notification_model.dart
| | |-- news/ArticleModel.dart
| | |-- request_history/request_history_model.dart
| | +--- work_schedule/work_schedule_model.dart
| |-- repositories/
| | |-- timesheet/timesheet_repository_impl.dart
| | |-- notification/notification_repository_impl.dart
| | +--- request_history/request_history_repository_impl.dart
| +--- sources/
| | |-- datastate.dart # DataSuccess<T> / DataFailed<T>
| | +--- status.dart
|
|-- domain/
| |-- entities/
| | |-- auth/user.dart
| | |-- timesheet/timesheet_entity.dart
| | +--- news/article_entity.dart
| |-- repository/
| | |-- timesheet/timesheet_repository.dart # abstract interface
| | |-- notification/notification_repository.dart
| | +--- news/article_repository.dart
| +--- usecases/
| | |-- usecase.dart # abstract UseCase<T, Params>
| | |-- get_timesheet.dart
| | |-- get_article.dart
| | |-- submit_adjustment_report_usecase.dart
| | +--- register_device_usecase.dart
|
|-- presentation/
| |-- auth/pages/
| | |-- sign_in.dart
```

```

| | +-- sign_up.dart
| | |-- bloc/
| | |-- article/remote/remote_article_bloc.dart
| | +-- timesheet/remote/
| | |-- remote_timesheet_bloc.dart
| | |-- remote_timesheet_event.dart
| | +-- remote_timesheet_state.dart
| |-- choose_mode/
| | |-- bloc/theme_cubit.dart # HydratedCubit<ThemeMode>
| | +-- bloc/locale_cubit.dart # HydratedCubit<Locale>
+-- pages/
| |-- main/main_screen.dart
| |-- home/home_page.dart
| |-- timesheet/timesheet_page.dart
| |-- service/service_page.dart
| |-- notification/notification_page.dart
| |-- profile/profile_page.dart
| |-- work_schedule/work_schedule_setup_page.dart
| |-- adjustment_report/adjustment_report_page.dart
| +-- request_history/request_history_page.dart
+-- services/
| |-- firebase_service.dart # Singleton: FCM + Analytics + Crashlytics
| |-- auth_service.dart # SharedPreferences token/session
| |-- network_service.dart # Connectivity + ValueNotifier<NetworkStatus>
| |-- localization_service.dart # Custom JSON-based i18n
| |-- analytics_observer.dart # NavigatorObserver -> Firebase Analytics
| |-- api_error_handler.dart # Show dialog theo HTTP status
| |-- navigation_service.dart # GlobalKey<NavigatorState>
+-- work_schedule_notification_service.dart # Local notification scheduler

```

2.2 Clean Architecture Layers

```

+-----+
| PRESENTATION LAYER |
| Pages / Widgets / BLoC / Cubit |
| (Flutter UI, chua co business logic) |
+-----+
| calls UseCase
+-----v-----+
| DOMAIN LAYER |
| Entities / Repository (abstract) |
| / UseCases |
| (Pure Dart, zero Flutter deps) |
+-----+
| implements
+-----v-----+
| DATA LAYER |
| Models / Repository Impl / API Services |
| (Dio, JSON parsing, SharedPrefs) |
+-----+

```

Lưu ý thực tế: Một số page (ví dụ: service_page, profile_page) có thể truy cập services trực tiếp thay qua usecase. Đây là technical debt cần refactor.

3. DESIGN PATTERNS CHI TIẾT

3.1 BLoC Pattern

Nguyên lý: UI dispatch Event -> BLoC xử lý -> emit State -> UI rebuild.

UI --add(Event)--> BLoC --emit(State)--> BlocBuilder (rebuild UI)

Các BLoC/Cubit trong project:

- RemoteTimesheetBloc | Bloc | Khong persist | Timesheet data + month navigation
- RemoteArticlesBloc | Bloc | Khong persist | News articles
- ThemeCubit | HydratedCubit | Co (JSON) | Dark/Light mode
- LocaleCubit | HydratedCubit | Co (JSON) | VI/EN language

3.2 Repository Pattern

```
// Domain: chi khai bao contract
abstract class TimesheetRepository {
  Future<DataState<TimesheetEntity>> getTimesheet(int year, int month);
}

// Data: implement contract
class TimesheetRepositoryImpl implements TimesheetRepository {
  final TimesheetApiService _apiService;
  TimesheetRepositoryImpl(this._apiService);

  @override
  Future<DataState<TimesheetEntity>> getTimesheet(int year, int month) async {
    try {
      final result = await _apiService.getTimesheet(year, month);
      return DataSuccess(result);
    } on DioException catch (e) {
      return DataFailed(e);
    }
  }
}
```

3.3 UseCase Pattern

```
// Base abstract class
abstract class UseCase<T, Params> {
  Future<DataState<T>> call({required Params params});
}

// Concrete usecase
class GetTimesheetUseCase implements UseCase<TimesheetEntity, GetTimesheetParams> {
  final TimesheetRepository _repository;
  GetTimesheetUseCase(this._repository);

  @override
  Future<DataState<TimesheetEntity>> call({required GetTimesheetParams params}) {
    return _repository.getTimesheet(params.year, params.month);
  }
}

class GetTimesheetParams {
  final int year;
  final int month;
  const GetTimesheetParams({required this.year, required this.month});
}
```

3.4 Singleton Pattern

```
class FirebaseService {
  FirebaseService._(); // private constructor
  static final FirebaseService instance = FirebaseService._(); // static singleton
  bool _initialized = false;

  Future<void> initialize() async {
    if (_initialized) return; // guard chong goi nhieu lan
    // ... init
    _initialized = true;
  }
}
```

```

class NetworkService {
  static final NetworkService _instance = NetworkService._internal();
  factory NetworkService() => _instance; // factory tra ve cung instance
  NetworkService._internal();
}

```

3.5 Observer Pattern (Analytics)

```

class AppFirebaseAnalyticsObserver extends RouteObserver<PageRoute<dynamic>> {
  final FirebaseAnalytics analytics;

  @override
  void didPush(Route<dynamic> route, Route<dynamic>? previousRoute) {
    super.didPush(route, previousRoute);
    if (route is PageRoute) {
      analytics.logScreenView(
        screenName: route.settings.name ?? 'unknown',
        screenClass: route.runtimeType.toString(),
      );
    }
  }
}
// Dung: navigatorObservers: [AppFirebaseAnalyticsObserver(analytics: ...)]

```

3.6 DataState Sealed Pattern

```

abstract class DataState<T> {
  final T? data;
  final DioException? error;
  const DataState({this.data, this.error});
}

class DataSuccess<T> extends DataState<T> {
  const DataSuccess(T data) : super(data: data);
}

class DataFailed<T> extends DataState<T> {
  const DataFailed(DioException error) : super(error: error);
}

---

```

4. STARTUP FLOW - PHAN TICH TUNG DONG

4.1 So do startup

```

main()
|
|-- WidgetsFlutterBinding.ensureInitialized()
|-- FlutterNativeSplash.preserve(binding) <- giu splash, khong man trang
|
|-- _initHydratedStorage() <- SEQUENTIAL (phai xong truoc)
|   |-- try: getApplicationDocumentsDirectory()
|   |-- fallback: getTemporaryDirectory()
|   +-- fallback: InMemoryStorage()
|
|-- Future.wait([...], timeout: 30s) <- PARALLEL
|   |-- FirebaseService.instance.initialize() timeout:20s
|   |-- initializeDependencies()
|   +-- NetworkService().init()
|
|-- Post-init (sequential):
|   |-- FirebaseService.instance.notificationApiService = sl<NotificationApiService>()
|   |-- AuthService.isLoggedIn()

```

```
|   +-- if loggedIn: FirebaseService.registerCurrentDevice()
|
|   -- FlutterNativeSplash.remove()
+-- runApp(MyApp(isLoggedIn: ...))
```

4.2 Tại sao HydratedStorage phải init tu trước?

ThemeCubit và LocaleCubit extend HydratedCubit. Khi BlocProvider(create: (_) => ThemeCubit()) được gọi trong MyApp.build(), HydratedCubit constructor sẽ ngay lập tức gọi HydratedBloc.storage.read(storageToken).

Neu storage == null -> null dereference CRASH.

Do đó: storage phải được init trước runApp() và trước khi HydratedCubit nào được instantiate.

4.3 Tại sao Firebase có timeout riêng 20s?

Tren iOS, Firebase.initializeApp() nội bộ gọi APNs để setup push notification environment.

Neu device không có internet hoặc APNs bị chặn (corporate firewall, airplane mode), call này có thể hang vô thời hạn -> splash màn hình trang mai -> app trải nghiệm tệ.

Solution: timeout 20s -> tiếp tục chạy app không có Firebase (graceful degradation).

4.4 HydratedStorage fallback chain

```
Future<void> _initHydratedStorage() async {
  if (kIsWeb) { ... return; }

  // Native: thu documents -> temp -> memory
  Directory? storageDir;
  try {
    storageDir = await getApplicationDocumentsDirectory();
  } catch (e) {
    try {
      storageDir = await getTemporaryDirectory();
    } catch (_) {}
  }

  if (storageDir != null) {
    try {
      HydratedBloc.storage = await HydratedStorage.build(storageDirectory: storageDir);
      return;
    } catch (_) {}
  }

  // Absolute fallback: state không persist nhưng app chạy bình thường
  HydratedBloc.storage = InMemoryStorage();
}
```

4.5 MyApp widget structure

```
MultiBlocProvider(
  providers: [
    BlocProvider(create: (_) => ThemeCubit()), // HydratedCubit
    BlocProvider(create: (_) => LocaleCubit()), // HydratedCubit
    BlocProvider(create: (_) => sl<RemoteArticlesBloc>().add(const GetArticles())),
    BlocProvider(create: (_) => sl<RemoteTimesheetBloc>()),
  ],
  child: BlocBuilder<ThemeCubit, ThemeMode>(
    builder: (_, themeMode) => BlocBuilder<LocaleCubit, Locale>(
      builder: (_, locale) => MaterialApp(
        theme: AppTheme.lightTheme,
        darkTheme: AppTheme.darkTheme,
        themeMode: themeMode, // ThemeCubit drives this
        locale: locale, // LocaleCubit drives this
```

```

navigatorObservers: [AppFirebaseAnalyticsObserver(...)],
localizationsDelegates: [AppLocalizations.delegate, ...],
home: NetworkStatusBanner(
  child: isLoggedIn ? const MainScreen() : const SigninPage(),
),
),
),
),
)
---

```

5. DEPENDENCY INJECTION (GetIt)

5.1 Registration order

```

// Thu tu quan trong: dependency phai duoc register truoc consumer

// 1. Dio instances
sl.registerSingleton<Dio>(Dio()); // generic dio (cho NewsApiService)
// thpDio: local variable, passed directly to constructors

// 2. API Services (leaf nodes)
sl.registerSingleton<LoginApiService>(LoginApiService(thpDio));
sl.registerSingleton<NewsApiService>(NewsApiService(sl<Dio>()));
sl.registerSingleton<TimesheetApiService>(TimesheetApiService(thpDio));
sl.registerSingleton<NotificationApiService>(NotificationApiService(thpDio));
sl.registerSingleton<AdjustmentReportApiService>(AdjustmentReportApiService(thpDio));
sl.registerSingleton<RequestHistoryApiService>(RequestHistoryApiService(thpDio));

// 3. Repositories (depend on API services)
sl.registerSingleton<TimesheetRepository>(<
  TimesheetRepositoryImpl(sl<TimesheetApiService>())
);
sl.registerSingleton<NotificationRepository>(<
  NotificationRepositoryImpl(sl<NotificationApiService>())
);

// 4. UseCases (depend on repositories)
sl.registerSingleton<GetTimesheetUseCase>(GetTimesheetUseCase(sl()));
sl.registerSingleton<SubmitAdjustmentReportUseCase>(<
  SubmitAdjustmentReportUseCase(sl())
);
sl.registerSingleton<RegisterDeviceUseCase>(RegisterDeviceUseCase(sl()));

// 5. BLoCs (depend on usecases)
sl.registerSingleton<RemoteTimesheetBloc>(RemoteTimesheetBloc(sl()));
sl.registerSingleton<RemoteArticlesBloc>(RemoteArticlesBloc(sl()));

```

5.2 Hot restart guard

```

Future<void> initializeDependencies() async {
  // Khi hot restart trong debug, main() chay lai nhung GetIt khong bi reset
  // -> neu khong co guard: registerSingleton se throw "already registered"
  if (sl.isRegistered<LoginApiService>()) {
    debugPrint('[DI] Already initialized - skip (hot restart guard).');
    return;
  }
  setupApiErrorConfigs();
  // ...registrations
}

```

5.3 Post-DI injection pattern

FirestoreService can NotificationApiService de gui FCM token len server,

nhưng cả hai được init song song trong Future.wait.

Giai pháp: inject sau khi cả hai đã xong:

```
// Sau Future.wait([Firebase.init, initDependencies, Network.init])
FirebaseService.instance.notificationApiService = sl<NotificationApiService>();
// Kiểu: setter injection / property injection (khác constructor injection)

---
```

6. NETWORKING - DIO + INTERCEPTORS + ERROR HANDLING

6.1 Dio Interceptor chi tiết

```
InterceptorsWrapper(
  // onRequest: tu dong gan Bearer token
  onRequest: (options, handler) async {
    final token = await AuthService.getToken();
    if (token != null && token.isNotEmpty) {
      options.headers['Authorization'] = 'Bearer $token';
    }
    handler.next(options);
  },

  // onResponse: bat 4xx (vi validateStatus tra true cho < 500)
  onResponse: (response, handler) {
    final statusCode = response.statusCode ?? 0;
    if (statusCode >= 400) {
      final skip = response.requestOptions.extra['skipErrorDialog'] == true;
      if (!skip) {
        String? serverMessage;
        try {
          final data = response.data;
          if (data is Map) {
            serverMessage = (data['Message'] ?? data['message'] ?? data['error'])?.toString();
          }
        } catch (_) {}
        ApiErrorHandler.handleHttpError(statusCode, serverMessage: serverMessage);
      }
      handler.reject(DioException(
        requestOptions: response.requestOptions,
        response: response,
        type: DioExceptionType.badResponse,
        message: 'HTTP $statusCode',
      ), false); // false = không gọi thêm onError interceptors
      return;
    }
    handler.next(response);
  },

  // onError: handle network errors
  onError: (error, handler) {
    final skip = error.requestOptions.extra['skipErrorDialog'] == true;
    if (!skip) {
      const networkErrors = {
        DioExceptionType.connectionTimeout,
        DioExceptionType.receiveTimeout,
        DioExceptionType.sendTimeout,
        DioExceptionType.connectionError,
      };
      if (networkErrors.contains(error.type)) {
        ApiErrorHandler.handleNetworkError();
      }
    }
    handler.next(error);
  },
);
```

)

6.2 API Error Config System

HTTP 401 -> force logout + navigate ve SigninPage (xoa stack)

HTTP 403 -> dialog "Khong co quyen truy cap"

HTTP 404 -> dialog "Khong tim thay du lieu"

HTTP 500 -> dialog "Loi may chu"

HTTP 503 -> dialog "Dich vu tam ngung"

Cau truc config:

```
class ApiErrorDialogConfig {
  final IconData icon;
  final Color iconColor;
  final String title;
  final String defaultMessage;
  final List<ApiErrorActionConfig> actions;
}

// Register cho 401
ApiErrorConfigs.register(401, ApiErrorDialogConfig(
  icon: Icons.lock_clock_outlined,
  iconColor: Color(0xFF57C00),
  title: 'Phien dang nhap het han',
  defaultMessage: 'Vui long dang nhap lai.',
  actions: [
    ApiErrorActionConfig(
      label: 'Dang nhap lai',
      onPressed: (ctx) async {
        Navigator.of(ctx).pop();
        await AuthService.logout();
        NavigationService.navigator?.pushAndRemoveUntil(
          MaterialPageRoute(builder: (_) => const SigninPage()),
          (_) => false,
        );
      },
    ),
  ],
));
```

6.3 Opt-out per request

```
// Login khong muon show dialog 401 (tu xu ly loi inline)
final response = await dio.post(
  '/api/account/internallogin',
  data: body,
  options: Options(
    extra: {'skipErrorDialog': true},
    validateStatus: (status) => status! < 600, // nhan ca 4xx/5xx
  ),
);

---
```

7. STATE MANAGEMENT - BLoC & HYDRATEDCUBIT

7.1 RemoteTimesheetBloc - Phan tich chi tiet

Events:

- GetTimesheet(year, month) - lay du lieu tu API hoac cache
- ChangeMonth(year, month) - doi thang hien thi
- SelectDay(selectedDate) - chon ngay de xem chi tiet
- RestoreTimesheetFromCache() - khi quay lai man hinh

States:

- TimesheetInitial() - trạng thái đầu, chưa có gì
- TimesheetLoading() - đang load lần đầu, hiện shimmer
- TimesheetRefreshing(timesheet) - đang load lại nhưng giữ UI cũ (có data cũ)
- TimesheetLoaded(timesheet, selectedDate) - có dữ liệu, hiện thi
- TimesheetError(error) - lỗi

Cache strategy:

In-memory cache: Map<String, TimesheetEntity>

Key format: "YYYY-M" (ví dụ: "2025-4", "2025-12")

Flow khi request tháng X:

1. Tháng tương lai? -> trả empty entity, KHÔNG gọi API
2. Cache hit? -> emit Loaded ngay (UX nhanh)
3. Cache miss + có data cũ? -> emit Refreshing -> gọi API -> emit Loaded/Error
4. Cache miss, không data? -> emit Loading (shimmer) -> gọi API -> emit Loaded/Error

Tại sao không dùng HydratedBloc cho Timesheet?

- Data attendance thay đổi theo ngày (cập nhật từ máy chấm công)
- Nếu persist -> user thay data cũ khi restart app -> confuse
- In-memory cache đủ cho UX flow trong cùng session
- Khi restart: force reload từ API để đảm bảo data mới nhất

```
// In-memory cache, key "year-month"
```

```
final Map<String, TimesheetEntity> _cache = {};
```

```
// Khi có data cũ: emit Refreshing giữ UI cũ
```

```
final prevLoaded = (state is TimesheetLoaded || state is TimesheetRefreshing)  
  ? state.timesheet  
  : null;
```

```
if (prevLoaded != null) {  
  emit(TimesheetRefreshing(timesheet: prevLoaded, selectedDate: state.selectedDate));  
} else {  
  emit(const TimesheetLoading());  
}
```

```
// Sau khi API xong
```

```
final dataState = await _getTimesheetUseCase(params: params);  
if (dataState is DataSuccess && dataState.data != null) {  
  _cache[_key(event.year, event.month)] = dataState.data!  
  emit(TimesheetLoaded(timesheet: dataState.data!, selectedDate: ...));  
} else if (dataState is DataFailed) {  
  emit(TimesheetError(dataState.error!));  
}
```

7.2 ThemeCubit (HydratedCubit)

```
class ThemeCubit extends HydratedCubit<ThemeMode> {  
  ThemeCubit() : super(ThemeMode.system);
```

```
  void updateTheme(ThemeMode themeMode) => emit(themeMode);
```

```
  @override
```

```
  ThemeMode? fromJson(Map<String, dynamic> json) {  
    return ThemeMode.values[json['theme'] as int];  
  }
```

```
  @override
```

```
  Map<String, dynamic>? toJson(ThemeMode state) {  
    return {'theme': state.index};  
  }  
}
```

HydratedCubit tu động:

Goi fromJson() khi khoi tao -> load state da luu
Goi toJson() moi khi emit state moi -> persist xuong storage

7.3 LocaleCubit

```
class LocaleCubit extends HydratedCubit<Locale> {  
  LocaleCubit() : super(const Locale('vi')); // default: Vietnamese  
  
  void changeLocale(Locale locale) => emit(locale);  
  
  @override  
  Locale? fromJson(Map<String, dynamic> json) => Locale(json['locale'] as String);  
  
  @override  
  Map<String, dynamic>? toJson(Locale state) => {'locale': state.languageCode};  
}
```

8. DATA LAYER - MODELS, REPOSITORIES, API SERVICES

8.1 Timesheet API

Endpoint: POST /api/employee/timeesheet
(Co typo "timeesheet" trong API goc - giong voi server)

Request:

Query params: ?employeeid={id}&year={year}&month={month}

Headers: Authorization: Bearer {token}

Body: (empty POST)

Response envelope (double-nested):

```
{  
  "status": 200,  
  "data": {  
    "status": 200,  
    "data": {  
      "employeeid": "EMP001",  
      "year": 2025,  
      "month": 4,  
      "dayOfWeek": 2,  
      "sumDayOfMonth": 30,  
      "timeSheetData": [  
        {  
          "DATE_WORKING": "2025-04-01T00:00:00",  
          "NGG": 0.0,  
          "NL": 0.0,  
          "P": 1.0,  
          "WD": 0.0,  
          "IS_DEFAULT": false,  
          "CHECK_POINT_LIST": [  
            {  
              "ID": 1,  
              "WORKING_DATE": "2025-04-01T00:00:00",  
              "EMPLOYEE_ID": "EMP001",  
              "TIME_IN": "08:30",  
              "TIME_OUT": "17:45",  
              "WD": 1.0,  
              "OT": 0.0  
            }  
          ]  
        }  
      ]  
    }  
  }  
}
```

```
}
}
```

IS_DEFAULT safe parse (server tra ve nhieu kieu):

```
// Server co the tra: true, false, "true", "false", 1, 0
bool _parselsDefault(dynamic value){
  if (value is bool) return value;
  if (value is String) return value.toLowerCase() == 'true';
  if (value is num) return value != 0;
  return false;
}
```

8.2 TimesheetEntity day du

```
class TimesheetEntity extends Equatable {
  final int year;
  final int month;
  final String employeeId;
  final int dayOfWeek; // weekday của ngày 1 trong tháng (1=Mon, 7=Sun)
  final int sumDayOfMonth; // số ngày trong tháng
  final List<TimeSheetDataEntity> timeSheetData;
}
```

```
class TimeSheetDataEntity extends Equatable {
  final DateTime dateWorking;
  final double? ngG; // Lam ngoai gio (Overtime loại 1)
  final double? ngG2; // Lam ngoai gio loại 2
  final double? nL; // Nghi le
  final double? bL; // Bu le
  final double? b; // Benh
  final double? p; // Phep nam
  final double? pr; // Phep riêng
  final double? ro; // Nghi không lương
  final double? sickLeave;
  final double? n; // Nghi không phep
  final double? tN; // Tai nan
  final double? hT; // Nghi hàng tuần (weekend)
  final double? ca3; // Ca 3
  final double? cDC; // Cach dieu chinh
  final double? o; // Nghi om
  final double? tS; // Thai san
  final double wd; // Working days fraction (0.0, 0.5, 1.0)
  final double? numHour;
  final double? numHourExtra;
  final String? note;
  final bool isDefault; // true = ngày không có data thực
  final List<CheckingPointEntity> checkingPoints;
}
```

```
class CheckingPointEntity extends Equatable {
  final int id;
  final DateTime workingDate;
  final String employeeId;
  final String? timeIn; // "HH:mm" string
  final String? timeOut; // "HH:mm" string
  final double wd; // working day value
  final double? ot; // overtime
}
```

8.3 Notification API

Endpoints:

```
GET /api/employee/getmessage?mode={mode}&page={page}&pagesize={size}
mode: 0=All, 1=Unread, 2=Read
-> NotificationListResponse { items: [], totalCount, unreadCount }
```

GET /api/employee/hasread?id={id}

-> mark notification as read

GET /api/employee/UnreadCount

-> { data: int }

POST /api/account/RegisterNotification

Body: { employeeId, fcmToken, deviceType: "Mobile" }

8.4 Login API

POST /api/account/internallogin

Body: { username: "xxx", password: "xxx" }

skipErrorDialog: true (login tu xu ly loi)

validateStatus: (s) => s! < 600 (nhân ca 4xx)

Response success:

```
{
  "status": 200,
  "data": {
    "token": "eyJ...",
    "username": "EMP001",
    "displayname": "Nguyen Van A",
    "email": "a@company.com",
    "position": "Developer",
    "department": "IT"
  }
}
```

8.5 Adjustment Report API

POST /api/employee/requestadjust

Headers: Authorization: Bearer {token}

Body: {

```
  "employeeId": "EMP001",
  "date": "2025-04-15",
  "timeIn": "08:30",
  "timeOut": "17:30",
  "reason": "Quen cham cong buoi sang"
}
```

9. FIREBASE - FCM, ANALYTICS, CRASHLYTICS

9.1 Firebase init sequence

// Trong FirebaseService.initialize():

1. Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform)

2. analytics = FirebaseAnalytics.instance

3. crashlytics = FirebaseCrashlytics.instance

4. messaging = FirebaseMessaging.instance

5. _initCrashlytics() -> set collection, FlutterError.onError, PlatformDispatcher.onError

6. _initAnalytics() -> setAnalyticsCollectionEnabled(!kDebugMode)

7. _initFCM() -> requestPermission, local notif setup, handlers, token

9.2 Crashlytics setup

// Bat Flutter framework errors (widget errors)

FlutterError.onError = crashlytics.recordFlutterFatalError;

// Bat async/platform errors (Zone errors)

PlatformDispatcher.instance.onError = (error, stack) {

crashlytics.recordError(error, stack, fatal: true);

return true; // true = xu ly xong, khong propagate

```
};

// Tat collection tren debug de khong lam nhieu data
await crashlytics.setCrashlyticsCollectionEnabled(!kDebugMode);

// Gan userId sau login
await crashlytics.setUserIdentifier(employeeld);
```

9.3 FCM 3 trang thai

Foreground (app dang mo):

```
FirebaseMessaging.onMessage.listen((message) {
  // iOS: setForegroundNotificationPresentationOptions xu ly -> KHONG dung _localNotifications.show()
  // Android: phai dung _localNotifications.show() de hien notification
  if (Platform.isAndroid) {
    _localNotifications.show(id, title, body, details);
  }
})
```

Background (app dang background):

```
FirebaseMessaging.onBackgroundMessage(_firebaseMessagingBackgroundHandler)
// Phai la top-level function (khong la method)!
// Ly do: isolate riêng, khong access class instances
```

Terminated (app bi tat):

```
final initialMessage = await messaging.getInitialMessage();
// Xu ly deep link / navigate khi app mo tu notification
```

9.4 iOS vs Android notification handling

iOS foreground: setForegroundNotificationPresentationOptions(alert:true, badge:true, sound:true)

-> Firebase SDK TU SHOW notification

-> KHONG goi _localNotifications.show() -> se bi duplicate!

Android foreground: FCM KHONG tu show

-> PHAI goi _localNotifications.show() tron foreground handler

9.5 FCM Token lifecycle

App start -> _initFCM() -> messaging.getToken()

-> Neu DI chua xong: skip (notificationApiService == null)

-> Sau DI xong: main() goi registerCurrentDevice()

Login thanh cong -> LoginScreen goi FirebaseService.registerCurrentDevice()

Token refresh -> messaging.onTokenRefresh -> registerCurrentDevice()

10. WORK SCHEDULE & LOCAL NOTIFICATIONS

10.1 WorkScheduleModel

```
class WorkScheduleModel {
  final List<WorkShiftEntry> shifts;
  final WorkScheduleReminder reminder;
}

class WorkShiftEntry {
  final String id; // UUID
  final String name; // "Ca sang", "Ca toi"
  final String checkInTime; // "08:00"
  final String checkOutTime; // "17:30"
  final bool crossesMidnight; // Ca qua dem (check-out < check-in)
```

```

final List<int> appliedDays;// [1,2,3,4,5] = Mon-Fri (ISO weekday)
final String repeatType; // "weekly", "daily", "custom"
final bool isActive;
}

class WorkScheduleReminder {
  final bool checkInEnabled;
  final bool checkOutEnabled;
  final int minutesBefore; // Nhac truoc bao nhieu phut
  final bool lateAlertEnabled; // Canh bao di tre (+5 phut)
  final bool overtimeAlertEnabled; // Canh bao lam them (+15 phut)
}

```

Persist: toJsonString() / fromJsonString() via SharedPreferences key "work_schedule".

10.2 Notification ID allocation

```

CHECK_IN_BASE = 1000 -> IDs 1000-1006 (7 ngay)
CHECK_OUT_BASE = 2000 -> IDs 2000-2006
LATE_BASE = 3000 -> IDs 3000-3006
OVERTIME_BASE = 4000 -> IDs 4000-4006

```

Mo moi shift x 7 ngay x 4 loi = toi da 28 notifications per shift

10.3 Scheduling flow

```

Future<void> scheduleFromWorkSchedule(WorkScheduleModel model) async {
  await _notifications.cancelAll(); // Cancel het notifications cu

  final now = DateTime.now();
  for (int dayOffset = 0; dayOffset < 7; dayOffset++) {
    final date = now.add(Duration(days: dayOffset));
    final isoWeekday = date.weekday; // 1=Mon, 7=Sun

    for (final shift in model.shifts) {
      if (!shift.isActive) continue;
      if (!shift.appliedDays.contains(isoWeekday)) continue;

      if (model.reminder.checkInEnabled) {
        final notifTime = parseTime(shift.checkInTime)
          .subtract(Duration(minutes: model.reminder.minutesBefore));
        final tzTime = tz.TZDateTime.from(notifTime, tz.getLocation('Asia/Ho_Chi_Minh'));

        if (tzTime.isAfter(tz.TZDateTime.now(_local))) {
          await _notifications.zonedSchedule(
            CHECK_IN_BASE + dayOffset,
            'Nhac check-in: ${shift.name}',
            'Ban co ca luc ${shift.checkInTime}',
            tzTime,
            _notifDetails('alert_sound'),
            androidScheduleMode: AndroidScheduleMode.exactAllowWhileIdle,
          );
        }
      }
      // Tuong tu cho checkOut, late (+5min), overtime (+15min)
    }
  }
}

```

Sound & vibration config:

```

AndroidNotificationDetails(
  'work_schedule_channel',
  'Work Schedule Reminders',
  importance: Importance.max,
  sound: RawResourceAndroidNotificationSound('alert_sound'), // res/raw/alert_sound.mp3
  vibrationPattern: Int64List.fromList([0, 500, 200, 500]),
),

```

DarwinNotificationDetails(sound: 'alert_sound.aiff')

11. AUTHENTICATION & SESSION

11.1 AuthService (SharedPreferences)

Keys:

```
'is_logged_in'    -> bool
'auth_token'      -> String (JWT)
'user_email'      -> String
'user_name'       -> String
'user_display_name' -> String
'employee_id'     -> String
'user_position'   -> String
'user_department' -> String
```

11.2 Login flow

User nhap username/password

```
-> SigninPage -> LoginApiService.login(username, password)
-> POST /api/account/internallogin (skipErrorDialog: true)
-> Success (200):
    AuthService.setLoggedIn(token, username, displayName, ...)
    FirebaseService.setUserId(employeeId) // Analytics + Crashlytics
    FirebaseService.registerCurrentDevice() // Register FCM token
    Navigator.pushReplacement(MainScreen)
-> Failure (401/400):
    Show error message inline (khong dung global dialog)
```

11.3 Logout (401 auto-logout)

1. Manual logout: ProfilePage -> AuthService.logout() -> navigate SigninPage
2. Auto logout (401): Dio interceptor -> ApiErrorHandler -> navigate SigninPage + clear stack
3. AuthService.logout(): SharedPreferences.clear() -> xoa tat ca

12. LOCALIZATION & THEME PERSISTENCE

12.1 Custom JSON-based i18n

```
class AppLocalizations {
  Future<bool> load() async {
    final jsonString = await rootBundle.loadString(
      'lib/i10n/${locale.languageCode}.json'
    );
    final Map<String, dynamic> jsonMap = json.decode(jsonString);
    _localizedStrings = jsonMap.map((key, value) => MapEntry(key, value.toString()));
    return true;
  }

  String translate(String key) => _localizedStrings[key] ?? key;
}

// Extension sugar
extension LocalizationExtension on BuildContext {
  String tr(String key) => AppLocalizations.of(this).translate(key);
}

// Usage trong widget
Text(context.tr('service_timesheet'))
```

l10n keys (vi du):

```
// lib/l10n/vi.json
{
  "service_title": "Dich vu",
  "service_timesheet": "Bang cong",
  "service_leave_request": "Xin nghi phep",
  "service_feedback": "Lich su yeu cau"
}

// lib/l10n/en.json
{
  "service_title": "Services",
  "service_timesheet": "Timesheet",
  "service_leave_request": "Leave Request",
  "service_feedback": "Request History"
}
```

12.2 Theme system

```
// Dark mode check
extension DarkModeHelper on BuildContext {
  bool get isDarkMode => Theme.of(this).brightness == Brightness.dark;
}
```

```
// Usage trong ServicePage
final isDark = context.isDarkMode;
color: isDark ? service.bgColorDark : service.bgColor
```

Fonts: BeVietnamPro (Regular, Medium, Bold, Light, Italic) + Satoshi
Duong dan: assets/fonts/BeVietnamPro-*.ttf

13. FEATURE INVENTORY DAY DU

13.1 Navigation structure

```
MainScreen (Bottom Navigation - 4 tabs)
|-- Tab 0: HomeScreen
| |-- Greeting + summary card
| |-- Quick links
| +-- News articles (RemoteArticlesBloc)
|
|-- Tab 1: TimesheetPage
| |-- Month navigation (<- Thang ->)
| |-- Summary cards (WD, P, ngG, etc.)
| |-- Calendar grid (30/31 days)
| |-- Day detail panel (checking points)
| +-- -> AdjustmentReportPage
|
|-- Tab 2: ServicePage
| |-- 6 service cards (2x3 grid)
| | |-- Bang cong -> TimesheetPage
| | |-- Xin nghi phep -> LeaveRequestPage
| | |-- Lich su yeu cau -> RequestHistoryPage
| | |-- Thanh tich (placeholder)
| | |-- So do to chuc (placeholder)
| | +-- Tai lieu (placeholder)
| +-- Info banner (beta notice)
|
+-- Tab 3: ProfilePage
  |-- User info (name, email, position, department)
  |-- Work schedule setup -> WorkScheduleSetupPage
  |-- Language toggle (VI/EN)
  |-- Dark mode toggle
```

13.2 ServicePage chi tiet

6 service cards trong 2x3 grid:

- service_achievement | ic_performance.svg | Color(0xFFFF97316) orange | placeholder
- service_leave_request | ic_leave.svg | Color(0xFF3B82F6) blue | -> LeaveRequestPage
- service_feedback | ic_message.svg | Color(0xFF8B5CF6) purple | -> RequestHistoryPage
- service_organization | ic_org.svg | Color(0xFF10B981) green | placeholder
- service_timesheet | ic_calender.svg | Color(0xFFEF4444) red | -> TimesheetPage
- service_documents | ic_file.svg | Color(0xFF0EA5E9) sky | placeholder

Dark mode: moi card co bgColor (light) va bgColorDark (dark) rieng.

13.3 NetworkStatusBanner

```
// Wrapper toan app
NetworkStatusBanner(child: isLoggedIn ? MainScreen() : SigninPage())

// Theo doi NetworkService.status (ValueNotifier<NetworkStatus>)
// Hien banner mau do "Mat ket noi mang" khi offline
// Tu an khi online tro lai
```

14. THIRD-PARTY LIBRARIES MATRIX (voi version & use case)

State Management:

- bloc 8.1.4 | BLoC core (event -> state)
- flutter_bloc 8.1.6 | BlocProvider, BlocBuilder, BlocListener
- hydrated_bloc 9.1.5 | Persist ThemeCubit, LocaleCubit qua app restart
- equatable 2.0.8 | Entities, states, events: equality comparison
- get_it 9.2.1 | Service Locator / Dependency Injection

Network:

- dio 5.9.2 | HTTP client voi interceptors (auth, error handling)
- retrofit 4.5.0 | Type-safe REST client generator (NewsApiService)
- http 0.13.6 | Base HTTP (retrofit dep)
- connectivity_plus 5.0.2 | Theo doi trang thai mang (WiFi/Mobile/None)
- pretty_dio_logger 1.4.0 | Debug logging cho Dio requests

Data/Storage:

- shared_preferences 2.2.3 | Token, user info, work schedule, selected month
- path_provider 2.1.4 | Get documents/temp directory cho HydratedBloc
- json_serializable 6.8.0 | Code gen cho @JsonSerializable models
- json_annotation 4.9.0 | Annotations (@JsonKey, @JsonSerializable)
- floor 1.5.0 | SQLite ORM (imported nhung chua co active module - DEBT)
- flutter_dotenv 6.0.0 | Load .env.dev / .env.prod

Firebase:

- firebase_core 2.32.0 | Firebase init
- firebase_messaging 14.9.4 | Push notification (FCM)
- firebase_analytics 10.10.7 | Screen tracking, custom events
- firebase_crashlytics 3.5.7 | Error monitoring & crash reporting

Notifications:

- flutter_local_notifications 16.3.3 | Local notification scheduling + FCM foreground (Android)
- timezone 0.9.4 | Timezone-aware scheduling (Asia/Ho_Chi_Minh)

UI:

- flutter_svg 2.0.9 | SVG icons trong service cards, navigation
- cached_network_image 3.3.1 | Cache ảnh từ network (articles)
- shimmer 3.0.0 | Loading skeleton effect
- flutter_native_splash 2.4.0 | Native splash screen config
- intl 0.18.1 | Date formatting, number formatting

Dev tools:

- build_runner | Code generation (json_serializable, retrofit)
- retrofit_generator | Generate Retrofit clients
- flutter_lints ^4.0.0 | Lint rules

15. PLATFORM CONFIG - ANDROID & IOS

15.1 Android (android/app/build.gradle)

```
compileSdk 36
targetSdkVersion 35
minSdkVersion flutter.minSdkVersion

// Multi-flavor
flavorDimensions "env"
productFlavors {
  dev {
    dimension "env"
    applicationIdSuffix ".dev" // com.company.app.dev
    versionNameSuffix "-dev"
  }
  prod {
    dimension "env"
    // applicationId giu nguyen
  }
}

// Java 8+ features (streams, lambdas) cho older Android
coreLibraryDesugaringEnabled true
multiDexEnabled true

apply plugin: 'com.google.gms.google-services'
apply plugin: 'com.google.firebase.crashlytics'
```

Auto-patch (android/build.gradle):

flutter_local_notifications 16.x: bigLargeIcon(null) -> ambiguous call trên API 35
 Android resolves sang wrong overload -> compile error
 Fix: subprojects patch explicit cast trong Java file

15.2 iOS (ios/Podfile)

```
platform :ios, '17.0'

post_install do |installer|
  installer.pods_project.targets.each do |target|
    # Fix firebase_messaging: non-modular import error
    if target.name == 'firebase_messaging'
      target.build_configurations.each do |config|
        config.build_settings['CLANG_ALLOW_NON_MODULAR_INCLUDES_IN_FRAMEWORK_MODULES'] = 'YES'
      end
    end
    # Tuong tu cho firebase_crashlytics
    flutter_additional_ios_build_settings(target)
  end
end
```

Van de iOS FCM (docs/IOS_FCM_FIX_GUIDE.md):

#import <Firebase/Firebase.h> -> non-modular header error khi build

Fix: CLANG_ALLOW_NON_MODULAR_INCLUDES_IN_FRAMEWORK_MODULES = YES

15.3 AppConfig (Environment)

```
class AppConfig {  
  static String get baseUrl => dotenv.env['BASE_URL'] ?? "";  
  static String get appTitle => dotenv.env['APP_TITLE'] ?? 'MyTHP';  
  static String get environment => dotenv.env['ENVIRONMENT'] ?? 'production';  
  static int get timeoutMs =>  
    int.tryParse(dotenv.env['TIMEOUT_MS'] ?? "") ?? 30000;  
}
```

.env.dev:

BASE_URL=https://dev-api.company.com

APP_TITLE=MyTHP Dev

ENVIRONMENT=development

TIMEOUT_MS=60000

.env.prod:

BASE_URL=https://api.company.com

APP_TITLE=MyTHP

ENVIRONMENT=production

TIMEOUT_MS=30000

16. CODE QUALITY & TECHNICAL DEBT

16.1 Diem manh

- Startup robust: timeout + 3-level storage fallback, không treo splash
- DI clean: đăng ký theo thu tu, hot restart guard
- Networking: interceptor tap trung, opt-out mechanism
- UX states: Loading/Refreshing/Loaded/Error day du
- FCM complete: Foreground/Background/Terminated + token refresh
- Localization: JSON-based, dễ thêm ngôn ngữ mới
- Theme persistence: HydratedCubit, survive app restart
- Work schedule: timezone-aware, 4 loại alert, 7 ngày ahead
- Crashlytics: bắt cả Flutter errors + async Zone errors

16.2 Technical Debt (nội thất trong phòng van)

API Key hardcoded | constants.dart | HIGH | Move to .env

Thiếu unit tests | test/widget_test.dart | HIGH | Add BLoC/UseCase/Repo tests

Page truy cập service trực tiếp | profile_page | MEDIUM | Thêm usecase layer

floor dep chưa dùng | pubspec.yaml | LOW | Remove hoặc implement

Widget test outdated | test/ | MEDIUM | Update/remove

16.3 Nếu interviewer hỏi "bạn sẽ làm gì tiếp theo?"

Priority 1: Unit tests

- UseCase tests: mock repository -> verify business logic
- BLoC tests: feed events -> assert states
- Repository tests: mock API -> verify mapping

Priority 2: CI/CD

- GitHub Actions: lint -> test -> build (dev + prod)
- Firebase App Distribution cho QA

Priority 3: Security

- Move newsAPIKey vào .env
- flutter_secure_storage thay vì SharedPreferences cho token

Priority 4: Architecture cleanup

- Chuan hoa tat ca pages qua usecase
- Remove floor dependency neu khong dung

17. INTERVIEW Q&A (30+ CAU)

Architecture

Q: Tai sao chon Clean Architecture?

A: Du an HR app se grow theo thoi gian (them module leave, payroll, org chart...). Clean Architecture cho phep moi team member lam doc lap tung layer. Domain layer khong phu thuoc Flutter -> de test thuan Dart. Presentation thay doi UI khong anh huong business logic.

Q: Khac gi MVVM?

A: Clean Architecture them UseCase layer giua ViewModel(BLoC) va Repository. UseCase giup tach biet ro business rule. MVVM thuong ViewModel truc tiep goi Repository -> coupling cao hon.

Q: Khi nao khong can UseCase?

A: CRUD don gian, khong co business logic phuc tap, team nho 1-2 nguoi, app scope nho. UseCase them boilerplate -> khong nen over-engineer.

BLoC

Q: Tai sao BLoC thay vi Provider/Riverpod?

A: BLoC buoc dinh nghia ro Event va State -> code de doc, audit. Test BLoC rat clean (feed event, assert state). Team lon: ai cung biet data flow.

Q: HydratedBloc la gi?

A: Extension cua BLoC giup persist state xuong storage (file JSON). Khi app restart, state duoc restore tu dong qua fromJson(). Dung cho ThemeCubit va LocaleCubit.

Q: Tai sao Timesheet khong dung HydratedBloc?

A: Data attendance co the thay doi trong ngay (cap nhat tu may cham cong). Neu persist -> user thay data cu khi restart. In-memory cache du cho UX flow trong session. Khi restart: force reload tu API de dam bao data moi nhat.

Q: Explain Refreshing state trong TimesheetBloc?

A: Khi user doi thang ma thang do chua co cache: thay vi show shimmer (mat UI cu), emit TimesheetRefreshing giu nguyen data thang truoc trong state. UI hien thi overlay loading nho. Khi API xong -> emit TimesheetLoaded voi data moi. UX muot ma hon.

Q: Khi nao emit Loading vs Refreshing?

A: Loading (shimmer): lan dau load, chua co data nao. Refreshing (giu UI cu): da co data cu, dang reload. Rule: neu state.timesheet != null -> Refreshing; otherwise Loading.

DI

Q: GetIt vs Provider?

A: GetIt la Service Locator, global access, khong can BuildContext. Provider/BlocProvider la InheritedWidget-based, can context. GetIt dung cho services/repositories/blocs. BlocProvider dung

cho state management trong widget tree.

Q: Hot restart guard là gì?

A: Khi Flutter hot restart, main() chạy lại nhưng GetIt.instance không bị reset (singleton trong process). Nếu registerSingleton lần 2 -> throw exception "already registered". Guard check isRegistered() trước khi register.

Networking

Q: Tại sao không dùng http package thay Dio?

A: Dio có interceptor system mạnh -> auto-attach token, tap trung error handling. http package không có interceptors -> phải implement thủ công ở mọi request.

Q: Giải thích validateStatus: (s) => s! < 500?

A: Mặc định Dio throw exception khi status >= 400. Với validateStatus < 500: 4xx về dưới dạng response thành công. Interceptor bắt trong onResponse (không phải onError). Lý do: muốn đọc response body để lấy server error message trước khi reject.

Q: skipErrorDialog hoạt động thế nào?

A: options.extra là Map<String, dynamic> cho phép pass metadata qua Dio call stack. Interceptor check extra['skipErrorDialog'] == true -> không show dialog. Dùng cho login (tự handle lỗi inline).

Firebase

Q: Background message handler phải là top-level function, tại sao?

A: Khi app bị killed, Flutter engine được khởi động lại trong isolate riêng cho background message. Isolate này không access class instances hay closures từ main isolate. Top-level function compile thành symbol có thể locate cross-isolate.

Q: Tại sao iOS foreground không dùng localNotifications.show()?

A: Firebase SDK trên iOS tự handle show notification khi foreground nếu setForegroundNotificationPresentationOptions được set. Nếu cũng gọi _localNotifications.show() -> user nhận 2 notification cho 1 push. Chỉ Android cần show() vì FCM Android không auto-show khi foreground.

Notifications

Q: Tại sao cần timezone package?

A: flutter_local_notifications schedule theo absolute time. Nếu user đổi timezone system -> notification van hiện dùng gio local. tz.TZDateTime dùng IANA timezone (Asia/Ho_Chi_Minh) để calculate absolute UTC time.

Q: Overlap notification IDs có xảy ra không?

A: Có khi nhiều shift. Base IDs: checkIn=1000, checkOut=2000, late=3000, overtime=4000. Day offset 0-6. Khi có nhiều shift, IDs có thể overlap. Potential bug: cần thêm shiftIndex vào ID calculation.

Auth/Security

Q: SharedPreferences có đủ secure cho JWT token không?

A: Không phải cách tốt nhất. SharedPreferences lưu plaintext. Trên non-rooted device đủ an toàn. Nên dùng flutter_secure_storage (Keychain/iOS, Keystore/Android) cho banking/healthcare app. App nội bộ HR -> acceptable risk.

Q: Logout hoạt động thế nào?

A: Manual: ProfilePage -> AuthService.logout() -> SharedPreferences.clear() -> navigate SigninPage.
Auto (401): Dio interceptor -> ApiErrorHandler -> navigate SigninPage + clear stack.

Performance/UX

Q: Shimmer là gì và khi nào dùng?

A: Skeleton loading effect (animation ảnh bạc chạy ngang). Dùng khi load data lần đầu (TimesheetLoading state). Tắt icon spinner vì cho user thấy layout structure trước. Không dùng khi Refreshing (có data cũ).

Q: GestureDetector ở root MaterialApp để làm gì?

A: Dismiss keyboard khi tap ra ngoài text field. Wrap toàn app với behavior:

HitTestBehavior.translucent -> catch taps mọi nơi -> FocusManager.instance.primaryFocus?.unfocus().

Tắt icon việc wrap từng page riêng.

Q: NetworkService dùng DNS lookup để làm gì?

A: connectivity_plus chỉ kiểm tra có kết nối WiFi/Mobile, không kiểm tra có internet thực sự. DNS lookup 'google.com' xác nhận kết nối internet thực. Tránh trường hợp: có WiFi nhưng không có internet (captive portal, mạng nội bộ).

18. FILE MAP QUAN TRỌNG

Phong van hoi ve: Doc file nay:

Startup / init lib/main.dart

Environment config lib/core/configs/app_config.dart

DI wiring lib/injection_container.dart

Networking/Error lib/injection_container.dart (Dio section)

lib/core/configs/api_error_config.dart

Timesheet BLoC lib/presentation/bloc/timesheet/remote/remote_timesheet_bloc.dart

Timesheet entity lib/domain/entities/timesheet/timesheet_entity.dart

Timesheet API lib/data/data_sources/remote/timesheet_api_service.dart

Firebase/FCM lib/services/firebase_service.dart

Auth/Session lib/services/auth_service.dart

lib/presentation/auth/pages/sign_in.dart

Notification API lib/data/data_sources/remote/notification_api_service.dart

Work schedule model lib/data/models/work_schedule/work_schedule_model.dart

Local notifications lib/services/work_schedule_notification_service.dart

Theme/Locale lib/presentation/choose_mode/bloc/theme_cubit.dart

i18n system lib/services/localization_service.dart

lib/l10n/vi.json, lib/l10n/en.json

Network monitor lib/services/network_service.dart

Analytics tracking lib/services/analytics_observer.dart

Hydrated storage fallback lib/utils/in_memory_storage.dart

Android build android/app/build.gradle

iOS Podfile patches ios/Podfile

19. CHECKLIST ON TAP

Kỹ thuật core (phải thuộc):

[] Về được số đo 3 layer Clean Architecture

[] Giải thích startup sequence từ main() đến runApp()

[] Giải thích tại sao HydratedStorage init trước Future.wait

[] Trình bày 5 states của TimesheetBloc và khi nào emit mỗi state

[] Giải thích Dio interceptor flow (onRequest -> onResponse -> onError)

[] Giải thích skipErrorDialog mechanism

[] Trình bày FCM 3 trạng thái (foreground/background/terminated)

[] Giải thích iOS vs Android notification handling khác nhau

[] Giải thích work schedule notification scheduling (7 days, 4 types, timezone)

Architecture decisions:

[] Tại sao BLoC thay vì Provider?

[] Tại sao GetIt thay vì InheritedWidget DI?

[] Tai sao in-memory cache cho Timesheet (khong persist)?

[] Tai sao timeout + fallback trong startup?

Technical debt (noi that):

[] Xac dinh duoc 4 technical debt chinh

[] De xuat duoc fix plan cu the cho moi debt

Bug scenario (chuan bi):

[] Neu notification khong hien tren iOS -> debug flow nao?

[] Neu FCM token khong duoc register -> check gi dau tien?

[] Neu Timesheet cache hien data cu -> tai sao va fix the nao?

[] Neu app crash khi dark mode -> debug HydratedBloc storage?

20. BUILD & RUN SCRIPTS

Development

flutter run --flavor dev -t lib/main_dev.dart

Production

flutter run --flavor prod -t lib/main_prod.dart --release

Build APK debug

flutter build apk --flavor dev -t lib/main_dev.dart --debug

Build APK release

flutter build apk --flavor prod -t lib/main_prod.dart --release

Build App Bundle (Google Play)

flutter build appbundle --flavor prod -t lib/main_prod.dart --release

iOS

flutter build ios --flavor prod -t lib/main_prod.dart --release

Code generation

flutter pub run build_runner build --delete-conflicting-outputs

Tests

flutter test

Analysis

flutter analyze

Ghi chu: Tai lieu nay duoc tong hop tu codebase thuc te.

Moi code snippet lay tu source files hien co.

Khi phong van: bam sat implementation thuc te, khong noi nhung gi chua implement.

Technical debt = trai nghiem thuc te, nen chia se thang than kem plan cai thien.